

Serverless/Event-driven Architectural Design Report

Swinburne university of Technology (Monday 4:30 PM) Melbourne, Victoria, Australia

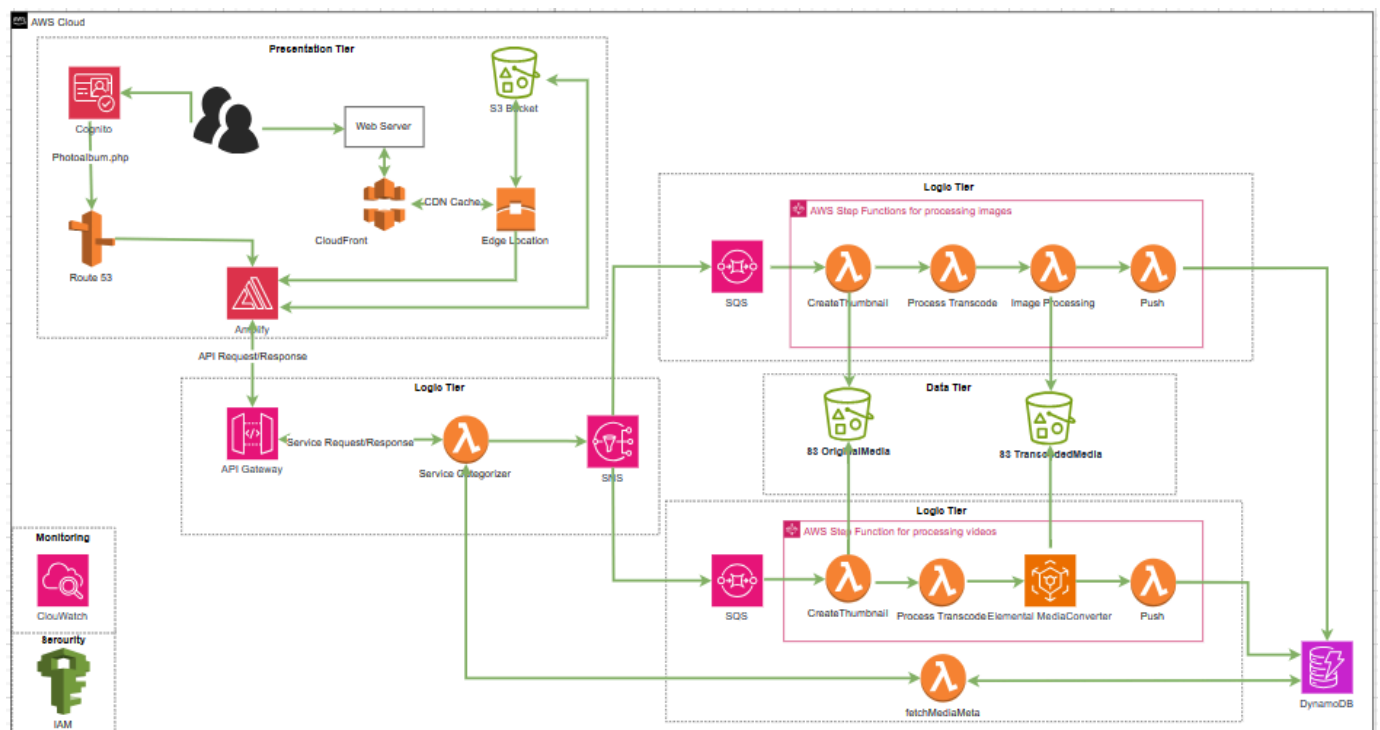
Harry Ward 103641884
Quang Vinh Le 104097488

Anabelle Lucarelli 104551544
Visal Noun 103793905

Introduction

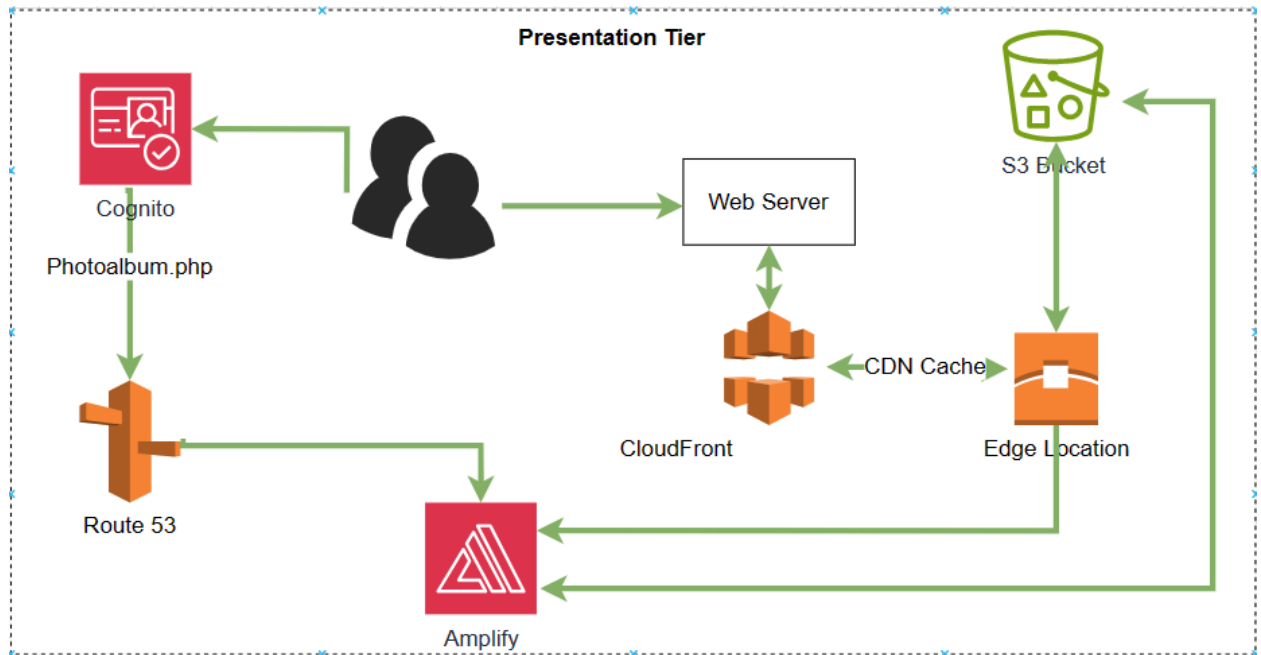
Our group is excited to present our proposal for the future enhancement of the successful Photo Album application to meet the growing demand that ensures scalability, cost-effectiveness, and improved global performance. Our proposed design leverages managed cloud services, adopts a serverless/event-driven architecture, optimizes media processing, and focuses on enhancing global response times while preparing for future functionalities such as video media handling. Below is our proposal;

ARCHITECTURE DESIGN



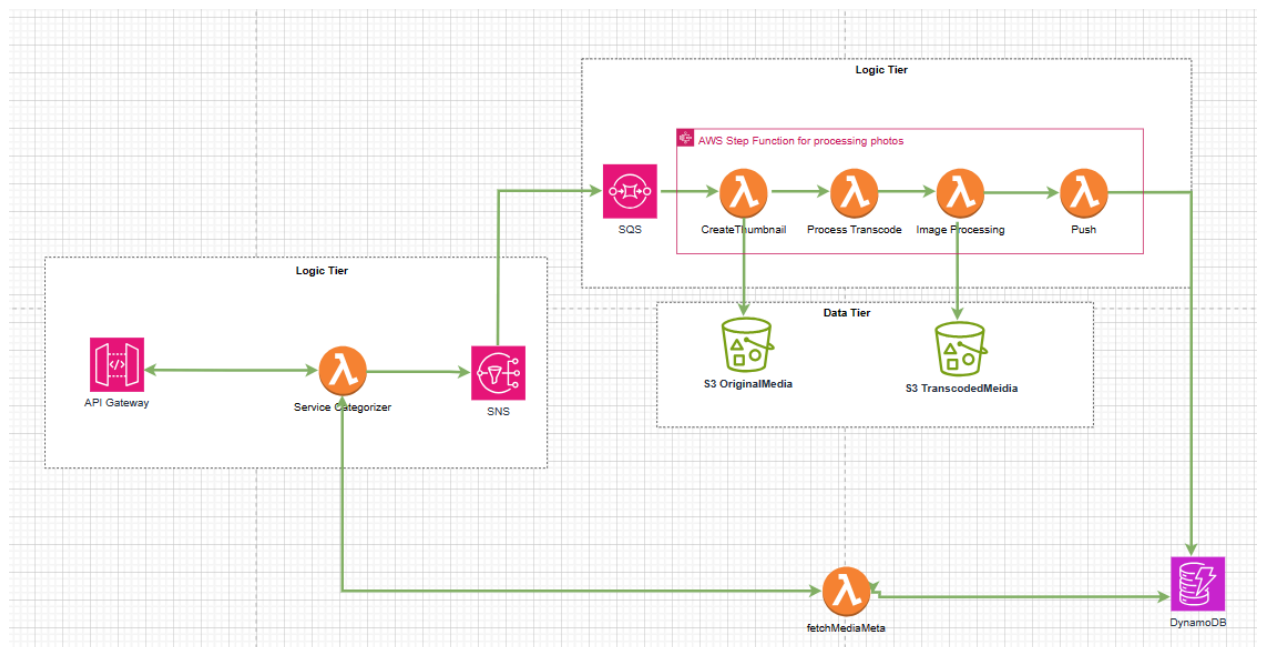
1. Content Delivery Mechanism

The proposed architecture leverages Amazon CloudFront as a Content Delivery Network (CDN) to optimize the delivery of static assets (web pages, images, and videos) from an S3 origin. CloudFront's global network of edge locations caches content closer to users, improving load times and reducing the burden on the origin server. Route 53 ensures requests are directed to the nearest edge location, while Amplify simplifies frontend deployment and integration with CloudFront.



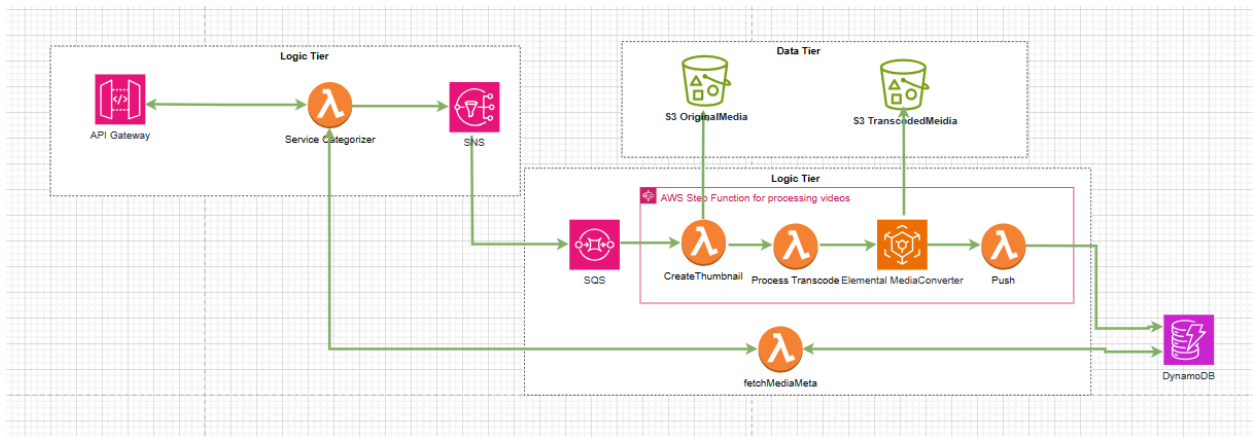
2. Upload and process photo

Users upload photos through the frontend, which are sent to an API Gateway. A Lambda function categorizes the file as an image and triggers an SNS notification. This notification is picked up by a worker Lambda, which adds the photo to an SQS queue. An AWS Step Functions workflow then orchestrates multiple Lambda functions to create thumbnails, process, and potentially transcode the image. The final images and their metadata are stored in S3 and DynamoDB, respectively.³

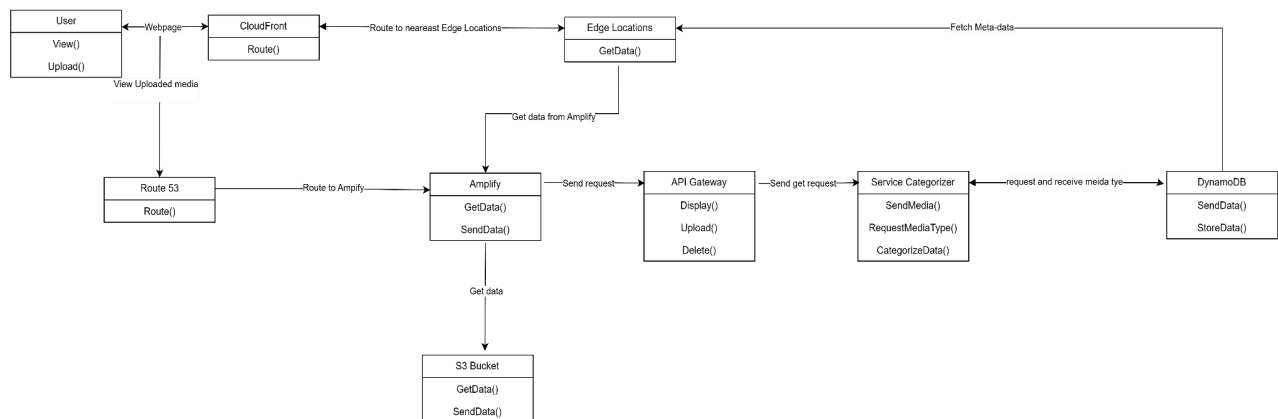


3. Upload and process video

Users upload videos through the frontend, triggering an API Gateway request. A Lambda function categorizes the file as video and sends a notification through SNS. A worker Lambda adds the video to an SQS queue, initiating an AWS Step Functions workflow. This workflow orchestrates Lambda functions to create thumbnails, perform initial processing, and utilize Elemental MediaConvert for transcoding. The processed videos and metadata are stored in S3 and DynamoDB, respectively.



1. Viewing Diagram



a. User Initiates Viewing:

- The user interacts with the "Webpage" (frontend) and performs a "View()" action, indicating they want to see a specific media item (photo or video).

b. CloudFront Content Delivery:

- The request is routed to "CloudFront," which is your content delivery network (CDN).
- CloudFront checks its cache at the nearest "Edge Location" to see if the requested media is already stored there.
- If the media is cached, CloudFront retrieves it ("GetData()") and sends it directly to the user. This is the fastest path, as it avoids fetching data from the origin server.

c. Fetching Metadata (if not in cache):

- If the media is not cached in CloudFront, a parallel process starts to fetch metadata:
 - CloudFront sends a request to "Amplify" to retrieve additional information about the media, such as its title, description, etc. ("Get meta from amplify").
 - CloudFront forwards the original request ("Route") to the origin server (S3 bucket) to retrieve the media file itself ("GetData()").

d. Retrieving from S3 and DynamoDB:

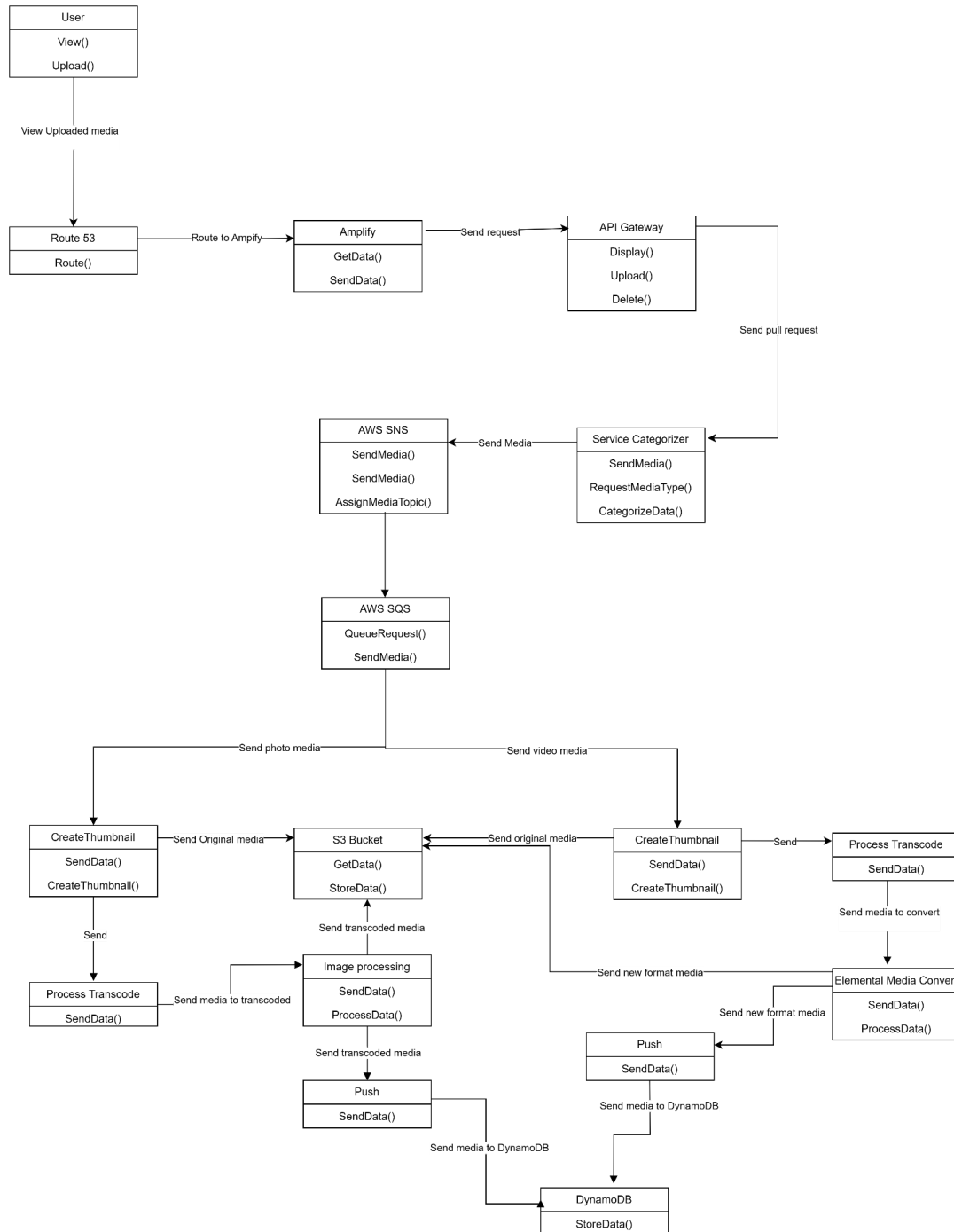
- The "Route 53" component is involved in DNS resolution, directing the request to the correct S3 bucket.
- The S3 bucket retrieves the requested media file and sends it back to CloudFront.
- While the media is being fetched, Amplify retrieves the metadata from DynamoDB ("GetData()") and sends it back to CloudFront.

e. Combining and Displaying:

- CloudFront combines the retrieved media file with the metadata.
- The combined data is then sent to the user's "Webpage," which displays the media along with its relevant information (title, description, etc.).

2. Uploading Diagram

The following UML sequence diagram illustrates the streamlined process of uploading and processing media (images and videos) within the Photo Album application. This process is designed to be highly automated, scalable, and efficient, ensuring minimal user wait times and accommodating the diverse range of media types users may upload.



Explanation of the Workflow:

- a. **User Initiation:** The user selects a media file (image or video) from their device and initiates the upload process through the frontend interface.
- b. **API Gateway and Service Categorization:** The frontend sends the media file to the API Gateway, which in turn invokes the "Service Categorizer" Lambda function. This function quickly determines whether the uploaded file is an image or a video.
- c. **Queueing and Notification:**
 - The categorized media file is placed in an Amazon Simple Queue Service (SQS) queue, which acts as a buffer to handle potential bursts of uploads and ensure smooth processing.
 - Simultaneously, a notification message is sent to the appropriate Amazon Simple Notification Service (SNS) topic, either for image processing or video processing, based on the file type.
- d. **Image Processing (If Image):**
 - If the media is an image, the SNS notification triggers a series of Lambda functions orchestrated by AWS Step Functions:
 - **CreateThumbnail:** Generates a smaller thumbnail version of the image for quicker previews.
 - **Process:** Applies any necessary image transformations (e.g., resizing, cropping).
 - **Image Processing:** Allows for additional image enhancements like watermarking or future AI-based analysis.
 - The processed image and its thumbnail are stored in the S3 bucket ("S3 Transcoded Media")
- e. **Video Processing (If Video):**
 - If the media is a video, the SNS notification initiates a video processing workflow, also managed by AWS Step Functions
 - **CreateThumbnail:** Generates a thumbnail image from the video.
 - **Process:** Performs initial video processing tasks (e.g., format validation).
 - **Transcode:** Utilizes AWS Elemental MediaConvert to transcode the video into different formats and resolutions for optimal viewing across devices.
 - The processed video, its thumbnail, and transcoded versions are stored in the S3 bucket.

f. Metadata Storage (DynamoDB):

- After successful processing, the relevant metadata for the media file (title, description, creationdate, keyword, reference ,etc.) is sent to DynamoDB for storage and retrieval.

DESIGN RATIONALE

Our design rationale focuses on leveraging managed cloud services on Amazon Web Services (AWS) and adopting a serverless/event-driven architecture to enhance the scalability, cost-effectiveness, and performance of your Photo Album application.

1.0 Managed Cloud Services Utilization:

We propose utilizing AWS managed cloud services to minimize in-house systems administration. Specifically, we suggest leveraging AWS Lambda for serverless compute, AWS Step Functions for orchestrating serverless workflows, and AWS S3 for media storage.

2.0 Scalability and Performance:

To accommodate the projected growth, we recommend migrating from t2.micro EC2 instances to a combination of serverless services and managed EC2 instances. By adopting a serverless/event-driven architecture, the system can automatically scale based on demand, ensuring optimal performance without exceeding performance limits.

3.0 Serverless/Event-Driven Architecture:

Embracing a serverless/event-driven approach will enhance the application's agility and scalability. AWS Lambda functions triggered by events such as S3 uploads will handle media processing tasks, ensuring efficient resource utilization and cost-effectiveness.

Additionally, through changing the database service Amazon RDS to Amazon DynamoDB, the service operates better in a serverless architecture to manage other applications like AWS Lambda and others, operating flexibly and on-demand capacity [7].

4.0 Media Processing Optimization:

Our proposed architecture includes a modular media processing pipeline that supports automatic generation of various media versions (e.g., thumbnails, low-resolution images, transcoded videos). By utilizing AWS Lambda, EC2 instances, and other managed services, we can distribute processing tasks efficiently while maintaining flexibility.

5.0 Global Performance Enhancement:

To address the issue of slow response times in countries outside Australia, we suggest leveraging AWS CloudFront for content delivery network (CDN) services. CloudFront will cache content at edge locations worldwide, reducing latency and improving user experience globally.

6.0 Security

A. Identity and Access Management (IAM)

To make our application have the right security, we propose using AWS Identity and IAM to control access to AWS resources. IAM enables us to manage permissions and roles to ensure that only the right people and services can access sensitive data[10].

- Role-based Access Control (RBAC): We can ensure users and services only have access to what they need by creating roles with specific permissions[10].
- IAM Policy: Allow us to create policies that set what each role can and can not do[11].
- Multi-Factor Authentication (MFA): We can add extra security, Requiring MFA for all IAM users. This requires the user to provide an alternative form of verification to gain access[12].

B. Amazon Cognito

We can use Amazon Cognito to manage user authentication to access control for the front end of the photo album[13].

- User Pools: Cognito User Pools will handle the user sign-up and sign-in process, providing tokens when users log-in. This will keep user access safe and secure[14].
- Identity Pools: Cognito Identity Pools will allow users temporary AWS credentials to access services like S3 directly, for uploading photos and videos[15].

7.0 Cost

Ideally, the application aims to be cost-effective through its implementation while also realistically approaching running the application long term with its budget. Through Amazon's own 'Pricing Calculator' [1], an estimated calculation budget we believe determines the appropriate cost for running the application long term and ensuring up to its requirements.

A. S3 Storage

With the application being solely storage focused over a long term period, the storage capacity for S3 will need to be able to accommodate photo and video files that, even with a policy to limit capacity of what can be uploaded, will still be extensive storage space. Including, processed content to be stored in a separate bucket.

Features required for S3 Storage are S3 Standard, S3 Object Lambda and Data Transfer.

Due to high traffic trends, a rough estimate of what would be appropriate storage capacity for S3 is roughly 500 GB per month. This capacity justifies space for not only original uploaded but for resolution change, thumbnails and more. No upfront costs exist for this feature, so 0.023 USD per GB multiplied by 500 GB the estimated amount is roughly 11.50 USD per month.

With extensive calculations from S3 Standard Feature (\$11.50), S3 Object Lambda (\$0.59) and Data Transfer feature (\$4.41 USD) the total monthly for one S3 Simple Storage system to run is \$16.50 USD. Since we are using two buckets for the architecture this number is doubled to match the architecture needs. Totalling the estimated cost to \$33.00 USD,

B. Lambda Functions

Through implementing multiple Lambda Functions that operate within the functionality of processing the uploaded content, the cost can vary and be very complex to make an appropriate estimate regarding its usage in the architecture.

The cost can relate to when it's executed in response to SNS and API Gateway implemented, which will be considered but not entirely able to be accurate in estimation [2].

Most features related to the lambda function include free tier. On average the duration of each provisioned request is 200 ms with allocated memory of 128 MB, with the invoke mode being buffered that doesn't cost extra.

Roughly costs \$18.50 USD to run per month approximately. With 9 different functions, the bare minimum cost for running all of them is \$166.50 USD every month. However, it is most likely more.

C. DynamoDB

To reduce costs when using a database for storing the photo and video files on the web servers, the Amazon RDS service is replaced with Amazon DynamoDB service. DynamoDB service offers capacity modes with specific billing processes known as on-demand and provisioned capacity modes. Since our services require on-demand with the current trend of traffic, on-demand capacity is required to fulfill these requirements.

This capacity mode specifically charges data reads and writes the capacity that helps with unpredictable application traffic. [8] A DynamoDB standard table usually charges 25GB first month free with \$0.25 USD for every GB afterward. Additionally, for backup and restoring features of the service, the service offers continuous backups or on-demand backup [9], continuous backups would be more efficient due to the large intake of traffic. Per GB a month, continuous backup is roughly \$0.20 USD for the database.

With extensive calculations, from Data Storage (\$25.00 USD), On-demand write setting (\$19.71 USD), On-demand read setting (\$32.85 USD), backup and restore (\$35.00 USD) and importing from Amazon S3 feature (\$15.00 USD), the monthly total for DynamoDB to run is \$127.56 USD.

D. CloudFront

AWS CloudFront is to be used across multi-regions as it is one of many services that will support a global network of edge locations and improve global response, in order to access other countries more efficiently not including Australia and the USA.

The first TB of data transfer is free for 10 million requests from all edge locations so this will be excluded from calculations.

Through each country you want to reach globally, calculations need to occur about data transfer out to the internet, data transfer out to origin and Number of request (HTTPS) for the month.

Assuming most customers using the application speak English, additional regional configurations are classified with Europe, Canada, Asia Pacific, South America and South Africa. Any other popular demographics outside these region zones will be added later through a review of trend analysis of traffic across regions. Roughly this will be estimated at \$191.40 USD.

E. Simple Notification Services (SNS)

Amazon Simple Notification Services are useful to promote notification clarification across Lambda functions in order to process its operations more efficiently with other AWS Services.

Through this FIFO topic would need to be implemented in order to notify Lambda Functions which item is being processed first and for what function.

Configuring the price of SNS requires calculating through service settings of requests, average message size and number of subscriptions. Additionally, configuring message filtering of data for every month.

Through calculating service settings (\$123.12 USD) and message filtering (\$0.09), the total estimated cost for this service to operate is \$123.21 USD.

F. Simple Queue Services (SQS)

Amazon Simple Queue Services is good to moderate the notification queue for the lambda functions to process the uploaded content more efficiently. Two SQS's are created, one for processing Image uploads and the other for video uploads.

Taking the number of SQS created, the cost will double per SQS configuration as they will be configured mostly the same in terms of calculating the price.

Calculations on the SQS depend on the Service settings regarding FIFO queue request and Data transfer for every month with inbound and outbound across the Internet, which is estimated to be 1TB for every 10 million requests monthly.

Through calculating the service settings (\$4.50 USD) and the Data transfer (\$92.16 USD), the total is estimated to cost roughly \$96.66 USD.

G. API Gateway

API Gateways help ensure data is accessible across all services in the architecture and make it easier for monitoring and maintaining from the presentation tier to the logic tier in the architecture diagram of our improved business proposal of the photo album application.

Calculating the cost to run an API gateway takes into fact the configuration of HTTP APIs and not other APIs as the other APIs don't really apply.

With HTTP API requests running roughly 10 Million estimated requests per month with an average size of each request being 512 KB.

The estimated cost of using API Gateway to run per month is \$10.00 USD.

H. Cognito

AWS Cognito is a security service that helps utilize the architecture using User pools and Identity pools. Calculating the cost to run Cognito in the presentation tier requires configuration of the Cognito User Pool and Machine 2 Machine information, as the Identify pool is no charge.

Through calculating the configuration of Cognito MAU (\$256.75 USD) and the Machine 2 Machine configuration (\$96.75 USD), the total estimated cost to run Cognito is \$353.50 USD.

I. AWS Step Functions

Implementing a serverless solution requires implementation of AWS Step Functions to help moderate the application more efficiently to the requirement needs.

With two step functions workspaces being implemented, the calculations will need to be doubled. To figure out standard workflow calculations per month is to recognise that the SQS Messages per month equate to the workflow requests received through the Step functions service.

With 10 Million requests estimated through each SQS to their own Step Functions, which will be 1 state transition will be required for each Step function to accommodate the SQS in each service.

Calculating the cost to run a singular Step Function will be roughly \$249.90 USD. Doubling that estimate to match the number of Step Function services and it comes to a total of \$499.80 USD.

J. Amplify

AWS Amplify receiving information from Route 53, S3 Bucket and Edge Location in the architecture. The calculation for Amplify requires configuring the Web App Hosting settings regarding; build minutes, data stored per month, data server per month, the number of Server-Side Rendering (SSR) requests and duration of each request. Through extensive calculations the total estimated cost is to be roughly \$15.22 USD per month.

K. Route 53

The use of Route 53 operates from the photo album's page with the calculations of Route 53 consider the Host Zones, DNS Failover Health Checks for endpoints, Route 53 Resolver and Route 53 Resolve DNS Firewall.

We only need one host zone within the Presentation tier, and other configuration settings can be defaulted to the minimum setting requirements to operate with communicating S3 bucket and other services. An estimated cost to run Route 53 is \$249.25 USD per month.

L. Elemental MediaConvert

AWS Elemental MediaConvert helps in the AWS Steps function service operating video formats that helps transcode the content uploaded in conjunction with the other Lambda function Image Processing to the TranscodedMedia Bucket.

Through configuration of the Output Configuration the total estimated cost of its usage is roughly estimated \$328.50 USD per month.

In total, for monthly costs to run the application with all the managed AWS features the total will be estimated to be \$1841.10 USD for our monthly Budget. For the requirements that expect the trend of traffic to continue for another 2-3 years, by multiplying the cost for another 24-36 months to total estimated cost long-term for the photo album application should be estimated around \$44,186.40 - \$66,279.60 USD, with roughly \$22,093.20 USD per year for the next 2-3 years.

CONCLUSION

In conclusion, our proposed design offers a scalable, cost-effective, and performance-driven solution to enhance your Photo Album application. By leveraging managed cloud services, adopting a serverless/event-driven architecture, optimizing media processing, and improving global performance, we are confident that our design will meet your current needs and prepare the application for future growth and functionality expansions.

We look forward to discussing our proposal further and collaborating with you to implement these enhancements for the application successfully.

REFERENCES

- [1] AWS, "AWS Pricing Calculator," *Calculator.aws*, 2019. <https://calculator.aws/#/>
- [2] "Serverless Computing – AWS Lambda Pricing – Amazon Web Services," *Amazon Web Services, Inc.* https://aws.amazon.com/lambda/pricing/?gclid=Cj0KCQjwxqayBhDFARIsAANWRnRqW451ge7XR4ro7JEkAHuAVZO4OIzs4oNMBbLLF3MuDI656mjNDREaAtFwEALw_wcB&trk=d4ab131e-73f1-4644-b31f-3a2cb50082ae&sc_channel=ps&s_kwcid=AL
- [5] AWS, "Amazon S3 Pricing," *Amazon Web Services, Inc.*, 2019. <https://aws.amazon.com/s3/pricing/>
- [6] "Amazon RDS vs DynamoDB: 12 Differences You Should Know," *Cloud Academy*, Oct. 13, 2022. <https://cloudacademy.com/blog/amazon-rds-vs-dynamodb-12-differences/>
- [7] "Amazon Web Services, Inc.," *Amazon Web Services, Inc.* https://pages.awscloud.com/GLOBAL-PAC-WEBINAR-AWS-OTT-FY21-08-24_2021_VW_s22e01-DAT_On-DemandVirtualWorkshop.html#:~:text=Amazon%20DynamoDB%20is%20a%20fast (accessed May 19, 2024).
- [8] "Amazon RDS vs DynamoDB: 12 Differences You Should Know," *Cloud Academy*, Oct. 13, 2022. <https://cloudacademy.com/blog/amazon-rds-vs-dynamodb-12-differences/>
- [9] "Amazon DynamoDB Pricing for On-Demand Capacity," *Amazon Web Services, Inc.* <https://aws.amazon.com/dynamodb/pricing/on-demand/>
- [10] "AWS Identity and Access Management: Amazon Web services INC." <https://aws.amazon.com/iam/>
- [11] "IAM Policies" *Amazon Web Services INC.* https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies.html
- [12] "Multi-Factor Authentication (MFA)" *Amazon Web Services Inc.* https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_mfa.html
- [13] "Amazon Cognito Overview" *Amazon Web Services Inc.* <https://aws.amazon.com/cognito/>
- [14] "User Pools" *Amazon Web Services Inc.* <https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-identity-pools.html>
- [15] "Identity Pools" *Amazon Web Services Inc.* <https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-identity.html>